DATA SCIENCE PRACTICUM 2026 · LESSON 05

Hub API Data BERT

Hugging Face: models and datasets in one line. LLMs through an API. BERT, and training our own classifier of Czech movie reviews.

Oct 16, 2026 · Faculty of Science, Masaryk University · 4 Colabs in lesson05/

HOMEWORK CHECK-IN

Assignment 02: how did it go?

Show of hands

Who submitted the dataset and the training notebook?

Who got the Streamlit app running?

What accuracy did you get? Which classes got mixed up?

What broke? Python version, model size, `requirements.txt`, Hugging Face login…?

Checklist

dataset on the Hub or GitHub
training notebook (GPU)
model in a Hugging Face model repo
Streamlit app (optional this week)

Show & tell

2–3 volunteers: open your app and let the class try it with their own pictures.

TODAY

Four notebooks, one running example

	notebook	what you will do
1	`01_gpt2_from_the_hub`	download GPT-2, generate text and data, log in, upload
2	`02_openai_api`	call an LLM through an API, pay per token, get structured data back
3	`03_datasets`	load, explore and share datasets; a look at tokenizers
4	`04_bert_classifier`	BERT vs GPT, and train our own classifier

Running example: 30,000 Czech movie reviews from ČSFD. The LLM rates them without training (2), we explore them (3), and we train a classifier on them (4).

No new homework this week.

1

GPT-2 from the 🤗 Hub

One line downloads a model. What is behind it, how to generate text and data with it, and how to upload your own work.

Colab: 01_gpt2_from_the_hub

HUGGING FACE

The “GitHub of machine learning”

The **Hub** (huggingface.co) hosts models, datasets and apps (Spaces). Each one is a **git repository** with a web page: a model card, the files, the history, downloads.

A repository is called `owner/name`, e.g. `openai-community/gpt2`, `google-bert/bert-base-uncased`.

You already used it: the model repository for the Streamlit app in Assignment 02.

library	what it does
`transformers`	model classes, *pipeline()*
`huggingface_hub`	login, upload, download
`datasets`	load, filter, share data
`tokenizers`	fast tokenizers (Rust)

DOWNLOADING A MODEL

What does from_pretrained download?

```
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained(
    "openai-community/gpt2")
```

`AutoModel...` reads `config.json`, sees `"model_type": "gpt2"` and picks the right class, `GPT2LMHeadModel`.

Files land in a cache (`~/.cache/huggingface`), so the second load is instant.

6.2 GB

in the repository: the same weights for PyTorch, TensorFlow, JAX, ONNX, TFLite and Rust

≈ 550 MB

downloaded: `config.json`, `model.safetensors` and the tokenizer files

GENERATING TEXT

How to pick the next token

Prompt: `The capital of France is` · GPT-2 small · 25 new tokens

setting	continuation
`do_sample=False` greedy: always the most likely	… the capital of the French Republic, and the capital of the French Republic is the capital of the French Republic.
`temperature=0.7`	… a place that has never had much of a progressive left, the kind that has been the site of the civil rights movement and
`temperature=1.5`	… a place that's pretty close…
`top_k=50` only the 50 most likely tokens	… a place that has never had much of a progressive left, the kind that the far left would never have been able to win

The easy way: `pipeline("text-generation", model=...)` = tokenize → `generate` → decode.

GENERATING DATA

From a language model to a dataset

```
rows = []
for prompt in prompts:
    for r in generator(prompt,
            num_return_sequences=5,
            do_sample=True, temperature=0.9):
        rows.append({"prompt": prompt,
                     "text": r["generated_text"]})
df = pd.DataFrame(rows)
```

Synthetic data: as many texts as you want, e.g. to train or test a smaller model.

GPT-2 writes mostly nonsense. Today people use much stronger models for this (notebook 2), but the workflow is the same.

Next: upload the table to the Hub. For that we need to log in.

LOGGING IN

On a remote machine: an access token

You need to log in to **upload**, to download **private** or **gated** models (Llama, Gemma: accept the license first), and for higher download limits.

1. huggingface.co/settings/tokens → *Create new token* → *Write* → copy `hf_...`

2. Colab 🔑 *Secrets* → name `HF_TOKEN` → paste → *Notebook access* on

3. Done. `huggingface_hub` finds it automatically, in every notebook. Fallback: `login()`

```python
from huggingface_hub import whoami

user = whoami()["name"]
print("logged in as", user)
```

A token is a password

Never write it into a cell. Notebooks get shared and pushed to GitHub, and bots scan GitHub for tokens within minutes.

UPLOADING

push_to_hub: one line up, one line down

Data: our generated texts

```
from datasets import Dataset

Dataset.from_pandas(df).push_to_hub(
    f"{user}/gpt2-generated")
```

→ a page with a dataset viewer. Anyone can load it with `load_dataset("you/gpt2-generated")`.

A model you trained (e.g. in notebook 4)

```
model.push_to_hub(f"{user}/my-model")
tokenizer.push_to_hub(f"{user}/my-model")

# and anyone can:
from_pretrained("you/my-model")
```

`private=True` if others should not see it.

2

LLMs through an API

The biggest models do not fit on Colab, and their weights are not public. You send text, get text back, and pay per token.

Colab: 02_openai_api

OPENAI API

A request is a list of messages

```python
from openai import OpenAI
client = OpenAI(api_key=userdata.get("OPENAI_API_KEY"))

response = client.chat.completions.create(
    model="gpt-5.6-luna",
    messages=[{"role": "user",
               "content": "Explain transformers."}],
    reasoning_effort="low",
)
print(response.choices[0].message.content)
```

`choices[0].message.content`: the answer

`finish_reason`: `stop` or `length`

`usage`: tokens in and out, which is what you pay for

API key

In Colab Secrets as `OPENAI_API_KEY`, like `HF_TOKEN`. API billing is separate from ChatGPT: prepaid, at least $5.

PRICE AND REASONING

You pay for every token, even the hidden ones

gpt-5.6-luna	per 1M tokens
input	$0.20
output (incl. reasoning)	$1.20

The whole notebook costs about **1 cent**.

`reasoning_effort`: the model first writes hidden tokens with its reasoning. You see only how many.

`none` · `low` · `medium` (default) · `high` · …

Try it

A bat and a ball cost $1.10 in total. The bat costs $1.00 more than the ball. How much does the ball cost?

Compare `low` and `high`: the answer, and the number of hidden tokens.

With reasoning on, `temperature` and `logprobs` are not allowed.

CONVERSATION

The model has no memory

system	You are a grumpy Czech grandmother. Answer in Czech, at most three sentences.
user	What should I have for dinner?
assistant	(the first answer)
user	And what about pizza?

The **system** prompt sets the rules: role, style, language.

Every request starts from zero. A chat “remembers” because it sends the **whole history** every time.

So the input tokens grow with every turn: long chats get expensive.

Structured output: a Python object, not text

```python
class ReviewRating(BaseModel):
    stars: int      # 0-5
    sentiment: Literal["positive",
                       "negative", "mixed"]
    summary: str

r = client.chat.completions.parse(
    model=MODEL, messages=[...],
    response_format=ReviewRating)
r.choices[0].message.parsed.stars
```

20 Czech reviews from ČSFD, rated **without any training** (zero-shot). Then compared with the real stars.

Same client, other providers: change `base_url` (Hugging Face Inference Providers, Ollama on your laptop).

Coming up

In notebook 4 we go the other way: train our own small model on 2,000 labelled reviews.

3

🤗 Datasets

Load a dataset from the Hub in one line, look inside, filter and transform it, and share your own. Plus a quick look at tokenizers.

DATASETS

30,000 Czech movie reviews in one line

```
from datasets import load_dataset

reviews = load_dataset(
  "fewshot-goes-multilingual/cs_csfd-movie-reviews")

train = reviews["train"]   # 25,000 rows
train[0]                   # one row = a dict
train[:3]["comment"]       # a slice = lists
train.features             # typed columns
```

Splits: **train** 25,000 · **validation** 2,500 · **test** 2,500

Columns: the review, 0–5 stars, date, title, year, genres, directors, cast…

Ratings are **balanced**: about 4,200 reviews for each number of stars.

The **dataset viewer** on the Hub shows the data before you download it.

DATASETS

Not in memory: Arrow, filter, map

```
avatar = train.filter(
    lambda row: row["item_title"] == "Avatar")
# 54 reviews

def count_words(row):
    return {"n_words": len(row["comment"].split())}
train = train.map(count_words)   # a new column

df = train.to_pandas()
```

The table lives in an **Apache Arrow** file on disk. Rows are read only when you ask for them, so datasets bigger than your RAM work.

Also: `select`, `shuffle`, `sort`, `train_test_split`.

Most reviewed title: *Ordinace v růžové zahradě* (61 reviews).

DATASETS

Huge datasets and your own data

Streaming

```
wiki = load_dataset("wikimedia/wikipedia",
    "20231101.cs", split="train",
    streaming=True)
for article in wiki.take(3): ...
```

Czech Wikipedia: 534,044 articles, about 1 GB. Streaming reads the first ones right away, without downloading everything. This is how LLMs are trained on trillions of tokens.

Your own

```
mine = load_dataset(
    f"{user}/gpt2-generated")
avatar.push_to_hub(
    f"{user}/csfd-avatar")
```

Local files: `load_dataset("csv", data_files=...)`. On the Hub, datasets are stored as **Parquet**.

TOKENIZERS

The Czech token tax, again

GPT-2 · 24

`T` `enh` `le` `␣film` `␣by` `l` `␣nap` `rost` `á` `␣n` `uda` `,` `␣ale` `␣her` `ci` `␣hr` `á` `li` `␣sk` `v` `<0xC3>` `<0xBD>` `le` `.`

mmBERT · 18

`␣Ten` `hle` `␣film` `␣byl` `␣nap` `rost` `á` `␣n` `uda` `,` `␣ale` `␣her` `ci` `␣hrá` `li` `␣skvě` `le` `.`

on all 25,000 reviews	tokens per word
GPT-2 (50,257 tokens, English)	3.3
mmBERT (256,000 tokens, 1,800 languages)	2.0

Every `AutoTokenizer` runs on the Rust library `tokenizers`: both tokenizers on 25,000 reviews took about **2 s**, with `map(batched=True)`.

Colab § 6 · ␣ = space · <0xC3> = one byte of a Czech letter

4

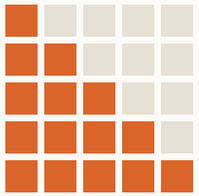
BERT, and our own classifier

Encoders vs decoders, filling in blanks, ModernBERT and mmBERT. Then we fine-tune mmBERT to tell positive Czech reviews from negative ones.

Colab: 04_bert_classifier

BERT

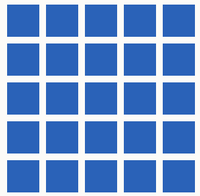
Encoder vs decoder: one mask



GPT · decoder

`The ?`

Each token sees only the past. Predicts the next token: **writes** text.



BERT · encoder

`The [MASK] barked at the mailman.`

Every token sees every token. Fills in blanks: **understands** text.

Otherwise almost twins: 12 blocks × 768 × 12 heads; GPT-2 124M parameters, BERT-base 110M. BERT (Google, 2018) is trained by hiding 15% of the tokens behind **[MASK]** and guessing them: no labels needed.

FILL IN THE BLANK

What do BERT and ModernBERT guess?

sentence	BERT (2018)	ModernBERT (2024)
`Paris is the [MASK] of France.`	`capital` 99.7%	`capital` 98.9%
`The [MASK] barked at the mailman.`	`man` 14.7% `dog` 14.2%	`dog` 80.1% `dogs` 6.8%
`return a [MASK] b`	–	`+` 98.8%

Bias

`The nurse said that [MASK] would be late.` → she 48%, he 18%

`The doctor said that ...` → he 34%, she 30%

mmBERT (2025)

ModernBERT trained on 1,800+ languages:

`Praha je hlavní [MASK] České republiky.` → město 99.8%

MODERNBERT

BERT with six years of LLM lessons

	BERT-base (2018)	ModernBERT-base (2024)
training data	3.3B words: Wikipedia, books	2T tokens: web, code, science
context	512 tokens	8,192 tokens
positions	learned table	RoPE (rotary)
attention	all tokens, every block	every 3rd block all tokens, the others only the 128 nearest
size	12 blocks, 110M	22 blocks, 150M

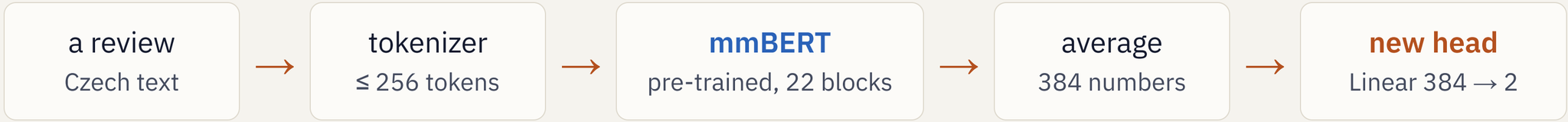
mmBERT

The ModernBERT architecture, 3T tokens in 1,800+ languages, including Czech. A Gemma tokenizer with 256,000 tokens.

We fine-tune **mmBERT-small**: 140M parameters.

FINE-TUNING

A pre-trained model plus a new head



Data: ČSFD reviews with 0–1 stars = **negative**, 4–5 stars = **positive**. 2,000 for training, 500 for testing.

Just like the image models in lessons 2 and 3: keep what the network learned, add a small layer for our task.

```
model = AutoModelForSequenceClassification\
  .from_pretrained("jhu-clsp/mmBERT-small",
      num_labels=2)
```

TRAINER

The training loop, written for us

```
args = TrainingArguments(
    output_dir="csfd-classifier",
    num_train_epochs=2, learning_rate=5e-5,
    per_device_train_batch_size=16,
    eval_strategy="epoch",
    fp16=True, report_to="none")

trainer = Trainer(model=model, args=args,
    train_dataset=train_ds, eval_dataset=test_ds,
    processing_class=tokenizer,
    compute_metrics=accuracy)
trainer.train()
```

The same 5 steps as in lesson 2, plus batching, padding, evaluation after every epoch, fp16 on the GPU.

88%

accuracy on the 500 test reviews after 2 epochs (250 steps), in the dry run

TWO WAYS

LLM through an API vs our own BERT

	gpt-5.6-luna, zero-shot (notebook 2)	fine-tuned mmBERT (notebook 4)
training data	none, just instructions	2,000 labelled reviews
setup	an API key	a GPU for a few minutes
cost per review	a fraction of a cent	free
speed, size	a remote frontier model	140M parameters, runs anywhere
your data	leaves your computer	stays with you
flexibility	any task, change the prompt	one task

Try it: `pipeline("text-classification", model=model)` on your own reviews. Can you fool it with irony? Upload it with `push_to_hub`.

WRAP-UP

The map of today

	run it yourself (🤗 Hugging Face)	call an API
decoder writes text	**GPT-2** from_pretrained, generate, synthetic data, push_to_hub	**gpt-5.6-luna** messages, reasoning, structured output
encoder understands text	**mmBERT classifier** load_dataset, map(tokenize), Trainer, pipeline	**embedding APIs** texts → vectors for search and RAG (later in the course)

THIS WEEK

No new assignment

Finish your Streamlit app from Assignment 02, if it is not running yet.

Play with the notebooks: generate with another model from the Hub, rate other texts with the API, explore a Czech dataset, fine-tune on your own labels.

Reading: Hugging Face LLM course · How to generate text · The Illustrated BERT · ModernBERT and mmBERT blog posts